Spring features companion

Sreedhar Suswaram | Senior Software Engineer technical interview | October 2026

This is a focused companion to the interview study guide. It explains Spring Framework, Spring Boot, and Spring MVC in plain English, then gives short answers you can practice aloud. The examples use modern Spring Boot with `jakarta.*` imports; confirm the versions used by the interviewing team before discussing version-specific behavior. The code fragments illustrate the idea and omit imports and surrounding project setup.

Source and version notes: The linked Spring Framework and Spring Boot pages are official documentation checked on October 22, 2026. Their current pages may describe a newer release than the team's runtime. In particular, `jakarta.*` imports and record-based examples suit modern Spring Boot; older projects may use `javax.*` and older Java syntax. The precise default behavior of auto-configuration, endpoint exposure, and shutdown should be checked against the application's Spring Boot version and settings. For the interview, explain the concept first, then qualify version-specific details if asked.

1. The three layers to remember

Layer	Main job	Example
Spring container	Create application objects (beans), connect dependencies, manage scope and lifecycle	`PaymentService` receives a `PaymentRepository`
Spring Boot	Start and configure the application with sensible conditional defaults	Configure an embedded server and a database client
Spring MVC	Map HTTP requests to methods, read inputs, validate them, and write responses	`GET /payments/{id}` calls a controller method

A request in one sentence: The embedded server receives the HTTP request, filters run, Spring MVC finds the controller method, validation checks the request, the service applies business rules, the repository reads data, and an HTTP response goes back through the chain.

Interview answer to practice: "I separate responsibilities across the Spring container, Boot, and MVC. The container manages dependencies and lifecycle. Boot assembles the application and conditional configuration. MVC maps requests to controller methods. My controller handles the HTTP contract, while services and repositories handle business rules and persistence."

2. Component scanning and bean definitions

Component scanning

Plain English: A *bean* is an object created and managed by Spring. Component scanning searches selected packages for classes marked with component annotations and registers them as beans. Constructor injection then connects those beans.

`@SpringBootApplication` includes component scanning. By default, scanning starts at the package containing the application class and covers its subpackages. Put the application class above the packages it should scan, or configure scanning explicitly. A class outside the scanned packages can exist in the project but never become a bean.

```
package com.example.payments;

@SpringBootApplication
public class PaymentsApplication {
    public static void main(String[] args) {
        SpringApplication.run(PaymentsApplication.class, args);
    }
}

// com.example.payments.service.PaymentService is scanned.
// com.otherteam.LegacyClient is not scanned by default.
```

Interview answer: "Component scanning finds annotated classes in the configured package tree and registers them in the application context. If an injection fails, I first check whether the implementation is in the scan path and whether its conditions or profile are active."

Common trap: `new PaymentService(...)` creates an ordinary Java object; it does not ask Spring to manage that instance. Spring features applied around managed beans may not apply to manually constructed instances.

`@Component`, `@Service`, and `@Repository`

All three mark a class as a candidate for component scanning. `@Component` is general. `@Service` communicates business logic. `@Repository` communicates data access and can participate in Spring's persistence exception translation when the relevant infrastructure is configured. Choose the annotation that expresses the class's role; adding both `@Component` and `@Service` is unnecessary.

```
@Service
public class PaymentService {
    private final PaymentRepository repository;

    public PaymentService(PaymentRepository repository) {
        this.repository = repository;
    }
}

@Repository
public class PaymentRepository {
    // Database access belongs here.
}
```

Interview answer: "These are Spring stereotypes. They all register components, but `@Service` and `@Repository` make the role clear. I use constructor injection so required dependencies are visible and the class can be tested without a full container."

Nuance: A repository interface discovered by Spring Data is registered through its repository infrastructure; it is not the same mechanism as scanning a concrete `@Repository` class. Spring component scanning.

`@Configuration` and `@Bean`

Plain English: Use `@Configuration` for a class that declares how beans are built. Use `@Bean` on a method that returns an object Spring should manage. This is useful when configuring a library class you cannot annotate or when construction needs explicit settings.

```
@Configuration
public class ClientConfiguration {
    @Bean
    public Clock clock() {
        return Clock.systemUTC();
    }
}
```

Interview answer: "I use component scanning for my application classes and `@Bean` methods for explicit construction, especially third-party clients. I keep bean configuration separate from business logic so dependencies and settings are easy to inspect."

Important detail: A full `@Configuration` class normally protects calls between `@Bean` methods so they follow container semantics. With `@Configuration(proxyBeanMethods = false)`, a direct Java call is just a direct call. Prefer method parameters to request dependencies from Spring rather than relying on one `@Bean` method calling another. Spring `@Bean` documentation.

`@Qualifier` and `@Primary`

If Spring finds two beans of the same required type, it needs a selection rule. `@Primary` sets the preferred bean for an ordinary single-valued injection. `@Qualifier` narrows candidates at a particular injection point. An explicit qualifier is appropriate when the caller needs a specific implementation. Do not assume these annotations create the beans; they only help choose among registered candidates.

```
interface PaymentGateway { Receipt charge(Payment p); }

@Component("bankA")
@Primary
class BankAGateway implements PaymentGateway { /* ... */ }

@Component("bankB")
class BankBGateway implements PaymentGateway { /* ... */ }

@Service
class InternationalPaymentService {
    InternationalPaymentService(
            @Qualifier("bankB") PaymentGateway gateway) {
        this.gateway = gateway;
    }
    private final PaymentGateway gateway;
}
```

Interview answer: "When several implementations share a type, I use `@Primary` for a sensible default and `@Qualifier` where a particular implementation is required. I prefer descriptive qualifiers because they reveal intent better than depending on an incidental field name." Qualifier selection; primary selection.

3. Scope, lifecycle, and dependency problems

Singleton, prototype, and request scope

Scope	What it means	Good use	Trap
Singleton	One bean instance for each bean definition in one application context	Usually stateless services, repositories, clients	Shared mutable request data can race; singleton does not mean thread-safe
Prototype	A new instance each time the container is asked for the bean	A stateful helper with a short, explicit use	Injecting it once into a singleton gives that singleton one instance
Request	One bean instance per HTTP request in a web-aware context	Request-specific context	A singleton needs a proxy/provider to obtain the current request bean

Plain English: Scope answers *how long a Spring-managed instance lives and when a new one is obtained*. It does not control database transaction boundaries or make code safe for concurrent requests.

```
@Component
@Scope("prototype")
class ReportBuilder { /* mutable, short-lived state */ }

@Service
class ReportService {
    private final ObjectProvider<ReportBuilder> builders;
    ReportService(ObjectProvider<ReportBuilder> builders) {
        this.builders = builders;
    }
    Report build() {
        ReportBuilder builder = builders.getObject();
        return builder.finish();
    }
}
```

The `ObjectProvider` call asks Spring for a new prototype on each use. Request-scoped beans can use a scoped proxy or a provider. If no HTTP request scope is active, asking for a request bean is invalid. A prototype's destruction callbacks are generally not managed automatically after the container hands it to you, so its owner must clean up resources when appropriate.

Interview answer: "Spring beans are singleton by default within an application context. I keep service state local to each method call. If I need a fresh stateful helper, I request a prototype when needed; injecting one prototype into a singleton at startup does not create a new instance per method call." Bean scopes.

Initialization and destruction

Spring constructs a bean, injects dependencies, runs initialization callbacks, and later runs destruction callbacks when it closes the managed scope. Common lifecycle mechanisms are `@PostConstruct`, `@PreDestroy`, and explicit `initMethod`/`destroyMethod` on `@Bean`.

```
@Component
class CertificateStatusCache {
    @PostConstruct
    void initialize() { /* validate initial configuration */ }

    @PreDestroy
    void close() { /* release owned resources */ }
}
```

Interview answer: "I use initialization for lightweight validation or setup after dependencies are available, and destruction for releasing owned resources. For a web service, I also consider graceful shutdown so in-flight requests have time to finish. I avoid slow remote calls that make startup unreliable without a clear readiness strategy."

Nuance: A clean context shutdown runs these callbacks for managed beans; process termination without orderly shutdown may not. Prototype cleanup is a special case. Bean lifecycle.

Circular dependencies

Plain English: A cycle exists when A needs B to be constructed and B needs A to be constructed. With constructor injection, neither can be finished first. This often shows that responsibilities are mixed together.

Better design: If `PaymentService` needs `AuditService` and `AuditService` needs `PaymentService`, extract a smaller `PaymentReader` or publish an event after the core action. Keep the dependency graph directional. `@Lazy` or a provider can defer construction, but that may hide the design issue.

Interview answer: "I treat a circular dependency as a design signal. I inspect why the two services know about each other, then extract shared behavior or change the direction of the dependency. I use lazy resolution only when the cycle has a justified lifecycle reason." Dependency and cycle handling.

4. Spring Boot startup and configuration

`@SpringBootApplication` and auto-configuration

`@SpringBootApplication` combines application configuration, component scanning, and Boot auto-configuration. Boot examines the libraries on the classpath, existing beans, and configuration properties before creating many default beans. For example, it can configure a web server and application plumbing when the relevant web starter is present.

Back-off rule: Conditional auto-configuration often creates a default bean only when you have not already provided a matching bean. It is conditional, not a blanket "my bean always wins" guarantee. The exact condition matters: type, name, classpath, property, or another criterion.

How to investigate: Start the application with `--debug` to view the conditions report in logs, or inspect Actuator's conditions endpoint if it is included and appropriately exposed. Look for a positive or negative match and the reason. This is more reliable than guessing which auto-configuration ran.

Interview answer: "Spring Boot removes setup work through conditional auto-configuration. When behavior surprises me, I check the conditions report and the actual bean definitions. If I provide my own bean, I confirm which condition backs off and which defaults remain active." Auto-configuration and conditions report.

Profiles

Plain English: A profile enables configuration only in selected environments. You can mark a component or configuration class with `@Profile("dev")` and select an active profile using configuration such as `spring.profiles.active=dev`.

```
@Configuration
@Profile("local")
class LocalGatewayConfiguration {
    @Bean PaymentGateway paymentGateway() {
        return new SandboxGateway();
    }
}
```

Use a local or test implementation only where intended. A profile is not an authorization boundary or a secret store. For production, externalize credentials and check the effective active profile during startup and deployment.

Interview answer: "Profiles help me select environment-specific beans and settings, while keeping the application code consistent. I make the active profile visible in deployment configuration and verify that a local mock cannot accidentally be selected in production." Spring Boot profiles.

Configuration properties and configuration validation

`@ConfigurationProperties` binds a related group of settings to a typed object. This is easier to check than spreading many `@Value` expressions through the code. Add `@Validated` and Jakarta Bean Validation constraints to fail startup when required settings are missing or invalid. Register a properties class using configuration-properties scanning or an enabling annotation.

```
@ConfigurationProperties(prefix = "payments.client")
@Validated
public record PaymentClientProperties(
        @NotBlank String baseUrl,
        @Min(100) long timeoutMillis) {}

@SpringBootApplication
@ConfigurationPropertiesScan
class PaymentsApplication { /* main method */ }

payments.client.base-url=https://example.invalid
payments.client.timeout-millis=1500
```

The example uses a Java record, so it needs a Java version that supports records. In a Java 8 project, use an ordinary mutable properties class with getters/setters and validation annotations. If you use nested objects, apply cascading validation where needed. Avoid putting secret values into logs or error messages.

Interview answer: "For grouped configuration, I use typed configuration properties and validate them at startup. This catches a missing URL or an unsafe timeout before the service begins handling traffic. I reserve profiles for selecting environment-specific settings, not for storing secrets." External configuration and validation.

Embedded server, health endpoints, and graceful shutdown

Embedded server: A Boot web application can start with its server packaged into the application. A servlet-based starter typically includes Tomcat by default, and another supported server can be selected. `server.port` changes the listening port; web-server choice is a dependency/configuration decision, not a controller annotation.

Health: With Actuator, `/actuator/health` reports application health. Liveness asks whether this instance should be restarted. Readiness asks whether it should receive traffic. Avoid a liveness check that fails merely because a shared database is temporarily unavailable; restarting every instance can make an outage worse. Check which Actuator endpoints are exposed and protect sensitive endpoint details.

Graceful shutdown: On a proper shutdown signal, the server stops taking new requests and gives active requests a bounded time to finish. Set a timeout compatible with the deployment platform's termination window. For long-running jobs or message consumers, also coordinate their stop and checkpoint behavior; an HTTP grace period alone is insufficient.

```
server.port=8080
spring.lifecycle.timeout-per-shutdown-phase=20s
```

Interview answer: "Boot packages a server with the application and exposes operational features through Actuator. I use readiness to control traffic, liveness for true process failure, and graceful shutdown to give in-flight work time to complete. I verify the deployment platform gives the process at least that much termination time." Embedded servers; Actuator endpoints; graceful shutdown.

5. Spring MVC request handling

`@RestController` and mappings

`@RestController` marks a controller whose method return values normally become response bodies, commonly JSON. `@RequestMapping` can establish a base path or define a mapping; method-specific forms such as `@GetMapping` and `@PostMapping` also make the HTTP method explicit. Spring MVC binds a path segment with `@PathVariable`, a query value with `@RequestParam`, and a body with `@RequestBody`.

```
@RestController
@RequestMapping("/api/payments")
class PaymentController {
    private final PaymentService service;
    PaymentController(PaymentService service) {
        this.service = service;
    }

    @GetMapping("/{id}")
    PaymentResponse get(@PathVariable String id) {
        return service.getAuthorizedPayment(id);
    }
}
```

This example leaves authentication details out of the fragment. The service method's name reminds you that looking up a payment by ID must also enforce the caller's permission to access it.

Interview answer: "I make mappings explicit about path and HTTP method. The controller translates HTTP input into a service call and returns a response DTO. It does not own database logic or trust a resource ID as proof of access." Spring MVC request mapping.

DTOs and `@Valid`

A *DTO* is a data transfer object: a deliberate shape for API input or output. An input DTO allows only fields the client is permitted to send. An output DTO avoids accidentally exposing internal fields or database relationships. `@Valid` triggers Bean Validation for a suitable request body when a validation provider is configured.

```
public record CreatePaymentRequest(
        @NotBlank String accountId,
        @NotNull @DecimalMin("0.01") BigDecimal amount,
        @NotBlank String currency) {}

@PostMapping
ResponseEntity<PaymentResponse> create(
        @Valid @RequestBody CreatePaymentRequest request) {
    PaymentResponse result = service.create(request);
    return ResponseEntity.status(201).body(result);
}
```

The DTO does **not** contain fields such as `approved`, `role`, `tenantId`, or `createdBy` for a client to set. Derive identity and tenant from trusted authentication context. `@Valid` checks field constraints; it does not check that the caller owns an account, that funds are available, or that the request is safe to repeat. Those are business and authorization rules. Check how validation failures are handled and converted into consistent client errors.

Interview answer: "I use request and response DTOs to control the API contract. I validate input shape at the boundary, then enforce authorization and business invariants in the service. I avoid binding clients directly to persistence entities, especially for privileged or internal fields." MVC validation.

`@ControllerAdvice` and stable error responses

`@ControllerAdvice` applies cross-controller behavior, including exception handling. `@RestControllerAdvice` is convenient when handlers return response bodies. A handler should map a known problem to a stable HTTP status and an error code clients can rely on. Keep unexpected internal details out of the response, and log enough context for operators to diagnose the incident.

```
record ApiError(String code, String message) {}

@RestControllerAdvice
class ApiErrors {
    @ExceptionHandler(PaymentNotFoundException.class)
    ResponseEntity<ApiError> missing(PaymentNotFoundException e) {
        return ResponseEntity.status(404)
                .body(new ApiError("PAYMENT_NOT_FOUND",
                        "Payment was not found"));
    }
}
```

For validation errors, define an intentional 400 response; for an authorization failure, use a status and message consistent with the application's security policy; for an unexpected failure, return a safe generic 500 response and log the internal cause once at the right boundary. Do not return database errors, stack traces, payment secrets, or raw personal data to the client. Spring also supports structured `ProblemDetail` responses; choose one stable external contract and apply it consistently.

Interview answer: "I centralize error translation so clients receive stable codes and appropriate HTTP statuses. I preserve the internal cause for diagnosis, but I do not leak internal details in the response. I handle validation, missing resources, forbidden access, and dependency failures deliberately." Controller advice; exception handlers.

Filters and interceptors

Filter: A Servlet filter wraps requests and responses around the servlet chain. It can run before Spring MVC selects a controller. Good uses include correlation IDs, request/response wrapping, and integration with a security filter chain.

Interceptor: A Spring MVC interceptor runs around a mapped handler. It can use handler information for MVC-specific work, such as certain request measurements or locale handling. It is not the preferred sole security layer; Spring's documentation recommends Spring Security or a similar early filter-chain approach for security.

Order to remember: Request -> filters -> DispatcherServlet -> handler interceptor -> controller -> service -> repository; response and completion callbacks occur on the way out. The exact behavior can change with async dispatch and error paths, so test filters/interceptors under those conditions.

Interview answer: "Filters operate at the servlet boundary, while MVC interceptors operate around selected handlers. I use filters for cross-cutting request concerns and Spring Security for authentication and authorization. I use an interceptor when the behavior depends on the MVC handler." Servlet filters; MVC interceptors.

6. Data access and production API habits

Parameterized queries

Plain English: SQL text should stay separate from user values. A bound parameter supplies data to a prepared statement, so a quote in a user value is not interpreted as part of SQL syntax.

```
String sql = "SELECT id, status FROM payment " +
             "WHERE tenant_id = :tenant AND id = :id";
MapSqlParameterSource params = new MapSqlParameterSource()
        .addValue("tenant", tenantId)
        .addValue("id", paymentId);
return jdbc.query(sql, params, rowMapper);
```

`NamedParameterJdbcTemplate` handles named values. It does not make dynamically concatenated table names, column names, or sort expressions safe. For those SQL identifiers, select from a server-side allowlist. Always scope payment queries to the authenticated tenant and authorize the specific resource.

Interview answer: "I bind all user-provided values as query parameters and use an allowlist for any dynamic SQL identifier. I also include the tenant and access boundary in the query or service logic. Parameter binding prevents SQL injection through values; it does not replace authorization." Spring JDBC core.

Bounded pagination

Unbounded list endpoints can exhaust database, service, or client resources. Give page size a default and maximum, use a deterministic sort with a tie-breaker, and be clear whether the API returns a total count. Offset pagination is straightforward but can grow expensive or skip/duplicate items when rows are inserted during browsing. Keyset/cursor pagination is often better for a changing feed, but it needs a stable ordering key and a contract for the cursor.

```
int size = Math.max(1, Math.min(requestedSize, 100));
Pageable page = PageRequest.of(pageNumber, size,
        Sort.by("createdAt").descending()
            .and(Sort.by("id").descending()));
```

Before this code, validate that `pageNumber` is nonnegative and decide whether invalid or excessive sizes should be rejected or capped. A sort field coming from the client also needs an allowlist. The database query must still enforce tenant and authorization constraints. A `Page` may require a count query; a `Slice` can avoid that when you only need to know whether another page exists.

Interview answer: "I bound page size, use deterministic sorting, and make the count contract explicit. If the dataset changes frequently or deep offsets become costly, I consider cursor pagination. I never let a pagination parameter bypass tenant filtering." Spring Data pagination reference.

Intentional logging

Log events that help answer a production question: what operation failed, when, on which safe correlation ID, and at which boundary. Keep secrets, bearer tokens, passwords, card details, and unnecessary personal data out of logs. Avoid logging the same exception in every layer; choose a boundary with enough context and preserve the cause. Pair logs with metrics and traces for latency investigations.

Interview answer: "I log enough structured context to trace an operation without recording sensitive payment or identity data. I preserve correlation IDs across calls and log unexpected exceptions once at the boundary that can add useful context. Stable error codes help connect client reports with internal diagnostics."

7. One connected example: an authorized payment lookup

1. A request reaches the embedded server: `GET /api/payments/p-123`.
2. The security filter chain authenticates the caller; a correlation filter attaches a safe request ID.
3. Spring MVC selects the `@GetMapping("/{id}")` method and binds `p-123` to `id`.
4. The controller calls `PaymentService.getAuthorizedPayment(id)`.
5. The service determines the trusted tenant and caller permissions, then asks the repository for the payment.
6. The repository runs a parameterized, tenant-scoped query. It returns only the fields needed for the response.
7. The service maps the result to `PaymentResponse`. The controller returns JSON. If the record is absent or access is denied, advice or security handling returns the defined error contract.
8. Metrics and correlation-aware logs show the outcome without exposing sensitive data.

Professional version to memorize: "For a payment lookup, I keep the controller thin. The security layer authenticates the caller, the service checks the caller's permission for the account, and the repository uses a parameterized, tenant-scoped query. I return a response DTO with only approved fields. The error handler produces stable client codes, while internal logs retain a correlation ID for investigation."

8. Interview drills and fast recall

Question	Strong short answer
What does component scanning do?	It finds eligible annotated classes in configured packages and registers them as beans.
`@Component` versus `@Service` versus `@Repository`?	All register components; the latter two express business and data access roles.
`@Bean` versus `@Component`?	`@Bean` registers the result of a method; `@Component` registers a scanned class.
What if two beans share one interface?	Use an explicit qualifier for a required implementation or one primary default.
Is a singleton bean thread-safe?	No. Scope controls instance count; code and state management determine thread safety.
Why did a prototype seem shared?	It was injected once into a singleton; use a provider or suitable scoped lookup.
How do you resolve a circular dependency?	Change the dependency graph or extract a shared responsibility; do not hide the design issue by default.
What does Boot auto-configuration do?	It creates conditional defaults based on dependencies, settings, and existing beans.
How do you explain a missing Boot bean?	Inspect component scanning, active profiles, and the conditions report.
What is a profile?	A way to select environment-specific beans or settings; it is not a secret store.
What does `@Valid` guarantee?	It checks configured constraints on input; it does not authorize access or prove business correctness.
Why use DTOs?	They define the external contract and prevent unintended binding or exposure of internal fields.
Filter versus interceptor?	Filter surrounds the servlet chain; interceptor surrounds a Spring MVC handler.
Why use `@ControllerAdvice`?	To apply consistent exception translation across controllers.
Does `NamedParameterJdbcTemplate` secure dynamic column names?	No. Bind values; allowlist dynamic SQL identifiers.
Why bound page size?	To protect database and service resources and keep API behavior predictable.

Final 45-second answer: "In Spring, I use component scanning and explicit bean configuration to make dependencies clear. Boot adds conditional application setup, typed configuration, and operational features such as health checks and graceful shutdown. In MVC, I keep controllers focused on HTTP: map requests, validate DTOs, and return stable responses. Services enforce authorization and business rules; repositories run tenant-scoped, parameterized queries. I watch for scope mistakes, circular dependencies, unbounded pagination, and accidental exposure of internal fields."